阶段一：向量加法指令 vadd.vv 的执行过程

> 依据：XiangShan `16ae9ddcd` 源码 + 本机构建 + NEMU/RTL 实测。
>
> 证据：`evidence/stage1/vadd-nemu.log`、`vadd-rtl.log`、`vadd-lane-isolation.log`、`vadd-wave-cycle.txt`、`evidence/wave/`。

---

1. 机器码解读

`vadd.vv v3, v1, v2` 编码为 32 位字：

```
0211 01D7   =   0000 0010 0001 0001 0000 0001 1101 0111
```

按位段拆开：

字段	位	值	含义
`funct6`	`[31:26]`	`000000`	加法操作
`vm`	`[25]`	`1`	不使用掩码
`vs2`	`[24:20]`	`00001`	第一个源，`v1`
`vs1`	`[19:15]`	`00010`	第二个源，`v2`
`funct3`	`[14:12]`	`000`	OPIVV 子类
`vd`	`[11:7]`	`00011`	目的寄存器，`v3`
`opcode`	`[6:0]`	`1010111`	OP-V 分组

---

2. 环境配置

阶段一到阶段三跑的是同一套环境，这里一次配完。所有步骤收在 `scripts/provision-2204.sh`，可以对着脚本核，也可以直接跑。

2.1 系统与依赖

Ubuntu 22.04，8 核 / 32 GB 内存（重建 RTL 时更要紧的是内存，见 2.4）。

```
sudo apt-get update
sudo apt-get install -y build-essential git curl wget unzip \
    autoconf flex bison libfl-dev device-tree-compiler \
    python3 python3-pip libgoogle-perftools-dev numactl
sudo apt-get install -y openjdk-17-jdk
sudo apt-get install -y gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

香山用 mill 构建（Scala），需要 JDK 11 以上；裸机测试程序用 RISC-V 交叉工具链编译。

2.2 Verilator

香山要求 Verilator 5.x，系统源里的版本一般偏旧，源码编译：

```
git clone https://github.com/verilator/verilator -b v5.052
cd verilator && autoconf && ./configure --prefix=/usr/local
make -j"$(nproc)" && sudo make install
```

Verilator 5.050 之后需要 clang 18 及以上，装之前先看一眼 `clang --version`。

2.3 取仓库与工作区

```
git clone https://github.com/OpenXiangShan/xs-env.git
cd xs-env && source setup.sh
```

`setup.sh` 会拉取 XiangShan 与 NEMU，并导出 `XS_PROJECT_ROOT`、`NOOP_HOME`、`NEMU_HOME`。后面所有构建都在这个 shell 里做，换窗口要重新 `source`。

2.4 构建 RTL 与仿真器

```
cd "$NOOP_HOME"
make verilog -j"$(nproc)" JVM_XMX=16G     # 生成 build/rtl/*.sv
make emu     -j"$(nproc)" JVM_XMX=16G     # 生成 build/emu
```

这一步是整个流程里最慢的，两步各要一小时上下。`JVM_XMX` 按物理内存给：32 GB 机器给 16G，16 GB 机器给 10G 更稳。之后改了 RTL 只要重新 `make emu`，`build/rtl` 不用重生成。

2.5 构建 NEMU（两套配置）

```
cd "$NEMU_HOME"
make riscv64-xs-ref_defconfig && make -j"$(nproc)"   # difftest 参考模型 → build/riscv64-nemu-interpreter-so
make riscv64-xs_defconfig     && make -j"$(nproc)"   # 独立解释器     → build/riscv64-nemu-interpreter
```

两套产物分别给"协同仿真"和"NEMU 单机跑一遍"用，不要混。改 NEMU 头文件后要先 `make clean`——它的 Makefile 不跟踪头依赖。

2.6 目录约定

后面所有脚本都按这两个环境变量找东西，没设就默认 `$HOME/XiangShan`、`$HOME/NEMU`：

```
export XIANGSHAN_DIR="$NOOP_HOME"
export NEMU_DIR="$NEMU_HOME"
```

3. 指令解析

这条指令的语义：

```
def vadd_vv(vs1, vs2, sew=8, vlmul=1):
    for i in range(vl):                 # vl = 16 (SEW=8, VLEN=128)
        vd[i] = sext8(vs1[i]) + sext8(vs2[i])
```

分类上，它属于 **OP-V 分组 `opcode=1010111` 下的 OPIVV 子类（`funct3=000`）**：整数、向量-向量。

`vm` 位决定运算的范围。`vm=1` 时 16 个 lane 全部参与；`vm=0` 时每个 lane 是否写回由向量寄存器 `v0` 的对应位决定：

```
vm=1:  vd[0..15] = vs1[0..15] + vs2[0..15]        # 全部写
vm=0:  vd[i] = (v0.bit[i] == 1) ? vs1[i]+vs2[i] : <按 vma 策略处理>
```

`vma` 有两种策略：`ma`（mask agnostic）把未选中的 lane 写成全 1；`mu`（mask undisturbed）保留旧值。保留旧值就需要把目的寄存器的旧内容读进来——这就是译码阶段要声明三个向量源的原因（见 §6）。

本程序的汇编序列：

```
li       t5, 0x600                    # 准备把 mstatus.VS 写成 11 (打开向量状态)
csrs     mstatus, t5                  # 写入 mstatus: VS=11 表示向量单元 Dirty
li       t0, 16                       # vl = 16, 本次处理 16 个元素
vsetvli  t1, t0, e8, m1, ta, ma       # 设 vtype: SEW=8、LMUL=1、tail/mask 均为 agnostic
li       t2, 0x0101010101010101       # 构造“0x01 重复 8 次”; 64 位寄存器装不下 128 位, 靠
vmv 广播
vmv.v.x  v1, t2                       # v1 = 0x01 × 16
vmv.v.x  v2, t2                       # v2 = 0x01 × 16
vadd.vv  v3, v1, v2                   # 被测指令: v3 = v1 + v2 = 0x02 × 16
vse8.v   v3, (buf)                    # 把 v3 的 16 个字节存回内存, 供程序逐字节检查
```

实测输出（NEMU 与 RTL 一致）：

```
vadd.vv test
v3 bytes: 02 02 02 02 02 02 02 02
RESULT: PASS (all lanes == 0x02)
```

---

4. 裸机配置

在 NEMU 或 RTL 仿真器上跑向量指令之前，需要先把两处状态配置好。

1）设置 `mstatus.VS` 字段。 `mstatus` 的 `VS` 字段（bit 10:9）表示向量单元的状态：`00` 表示 Off，`01` 表示 Initial，`10` 表示 Clean，`11` 表示 Dirty。硬件在 `VS=Off` 时把任何向量指令视为非法。裸机程序启动时该字段是 `00`，所以第一条向量指令会直接触发非法指令异常。把 `0x600`（即 `VS=11`）写入 `mstatus` 即可打开：

```
li   t5, 0x600
csrs mstatus, t5
```

写 `11`（Dirty）而不是 `10`（Clean），是因为程序接下来会修改向量寄存器，状态已经不是干净的。

2）在汇编入口设置栈指针 `sp`。 C 函数序言会立即使用 `sp` 保存返回地址和临时寄存器。如果在 `main` 里才设 `sp`，序言已经先执行过了，会向地址 `0xfffffffffffffec` 写内存并触发 `invalid mem write`。因此 `sp` 必须在 `_start` 里、调用任何 C 函数之前设置：

```
_start:
  li   sp, 0x80100000      # 栈放在 0x80100000，避开代码段与数据段
  call kmain
```

---

5. 香山处理一条指令的流水线

香山是乱序超标量处理器，一条指令依次经过下列阶段（括号内为代码位置）：

阶段	英文	做什么
取指	Fetch	从 ICache 取指令块，经 FTQ、IBuffer 送入译码（`frontend/`）
译码	Decode	解析 opcode 与 funct 字段，产出 `fuType`、`fuOpType`、源/目的类型、是否读写等控制位（`backend/decode/`）
重命名	Rename	把架构寄存器映射到物理寄存器，消除假依赖（`backend/rename/`）
派发	Dispatch	按 `FuType` 把 uop 送进对应发射队列，并在 ROB 分配表项（`backend/dispatch/`）
发射	Issue	源操作数就绪后从发射队列发出（`backend/issue/`）
读寄存器	Register Read	从物理寄存器堆读出源操作数（`backend/regfile/`）
执行	Execute	在功能单元（FU）内完成运算
写回	Writeback	结果写回物理寄存器，并广播唤醒等待它的指令
提交	Commit	ROB 按序提交，架构状态生效；异常在此刻被处理

指令在流水线里是“乱序执行、按序提交”：执行可以提前，但只有提交之后结果才对外可见。

---

6. 译码（Decode）

译码器先按 `opcode` 选中向量译码表，再按 `funct3`、`funct6` 匹配具体表项。`vadd.vv` 对应的表项在 `backend/decode/VecDecoder.scala:191`：

```
VADD_VV -> OPIVV(FuType.vialuF, VialuFixType.vadd_vv, T, F, F)
```

`OPIVV` 的定义在同文件 `:30`：

```
case class OPIVV(fu, fuOp, vWen, mWen, vxsatWen,
                 uopSplitType = UopSplitType.VEC_VVV,
                 src1 = SrcType.vp, src2 = SrcType.vp, src3 = SrcType.vp)
```

逐项展开：

字段	取值	作用
`fu`	`FuType.vialuF`	指定由向量整数算术单元执行
`fuOp`	`VialuFixType.vadd_vv`	指定该单元内做加法
`vWen`	`T`	需要写向量寄存器
`mWen`	`F`	不写标量寄存器
`vxsatWen`	`F`	不更新饱和标志
`uopSplitType`	`VEC_VVV`	该指令拆成几个微操作（uop）送给 FU
`src1`、`src2`、`src3`	均为 `vp`	三个源都是向量寄存器

关于 `uopSplitType`：一条向量指令可能被拆成多个 uop 分批发给 FU，拆几个取决于数据通路宽度与向量长度。`VEC_VVV` 表示这是一条三向量源指令。这条指令在 128 位通路上只需一个 uop——波形上 `t=8568` 只出现一次 `in.valid`，一次就带完整 128 位源、一次就输出完整 128 位结果。

关于三个源：`src1` 是 `vs1`，`src2` 是 `vs2`，`src3` 是**目的寄存器的旧值**。第三个源存在的原因是 `vm=0` 且 `vma=mu` 时需要保留未选中 lane 的旧值（见 §3）。`vm=1` 时前两个源就够用，波形上读到的是：

```
src_0 = 0x01010101010101010101010101010101     (v1, 全 1)
src_1 = 0x01010101010101010101010101010101     (v2, 全 1)
```

---

7. `vsetvli` 设置两个状态的值

`vsetvli` 本身不做运算，它写 `vtype` 和 `vl` 两个状态：

状态	本例取值	作用
`vtype.vsew`	`0`（e8）	每个元素 8 位
`vtype.vlmul`	`0`（m1）	单寄存器，VLEN=128
`vtype.vta` / `vma`	`ta` / `ma`	tail 与 mask 元素置全 1
`vl`	16	本次处理 16 个 lane

这三项里有两项容易看不懂，单独说明。

「单寄存器」指 LMUL=1。 RVV 允许把多个向量寄存器拼成一组用（LMUL=2/4/8），也允许把一个寄存器切段用（LMUL=1/2、1/4）。`m1` 表示不分组：`vs1`、`vs2`、`vd` 各只占一个 128 位寄存器，16 个 byte lane 全在这一个寄存器里。

「tail 元素置全 1」指 `vl` 不到 `VLMAX` 时剩下的那一段怎么处理。 本例 `vl = 16`，正好等于 `VLMAX`（SEW=8、LMUL=1、VLEN=128 → 16 个元素），所以没有 tail 元素。假如 `vl = 4`，那么 lane 4 到 lane 15 就是 tail。`ta`（tail agnostic）表示硬件不保证这些位置的值——可以写全 1，也可以保留旧值。

「mask 元素置全 1」指 `vm=0` 时没被 `v0` 选中的那些元素。 `ma`（mask agnostic）同样允许二选一。

香山的实现是统一填全 1，见 `backend/fu/vector/Mgu.scala:101`：

```
mask := Mux(maskBits(i), vd(i), Mux(info.ma, 1.U, maskOldVdBits(i)))
```

被选中的 lane 取新值；`ma` 时填 `1`；`mu` 时保留旧值。所以程序里写 `ta, ma`，就意味着"没算到的位置都会变成全 1"，检查结果时不能把那些位置当成 0 来比对。

---

8. 重命名、派发、发射

重命名（Rename）： 架构寄存器 `v0`–`v31` 的编号会被映射到物理寄存器。这样做是为了消除写后写（WAW）和读后写（WAR）这类假依赖——两条指令写同一个架构寄存器时，后一条可以分配到另一个物理寄存器，不必排队等前一条。重命名同时维护 `BusyTable`，记录每个物理寄存器“是否还在计算中”。

派发（Dispatch）： 按 `FuType` 把 uop 送进对应的发射队列，`vialuF` 的 uop 进入向量算术队列。同时在 ROB 分配一个表项，用于后续按序提交和异常处理。

发射（Issue）： 发射队列等待源操作数就绪。就绪的判据是 `BusyTable` 中对应物理寄存器被清零（表示产出它的指令已经算出结果）。就绪后取出 uop 发出，随后读物理寄存器堆。

延迟 1 拍： FU 的延迟由配置决定（`FuConfig` 的 latency 字段），elaboration 时会打印成 `fuLatencyMaps`。`vialuF` 的延迟是 1 拍，含义是 uop 在第 N 拍进入 FU、第 N+1 拍结果有效。这个数字决定了发射队列何时可以释放该 uop、写回仲裁如何安排时序——发射后不能立刻认为结果可用，必须按配置的拍数等待。

---

9. 执行：`VIAluFix`

代码位置：

层	文件	说明
FU 外壳	`backend/fu/wrapper/VIAluFix.scala`（376 行，`class VIAluFix` 在 `:134`）	接线、掩码合并、异常与 `vxsat`
算术核心	`yunsuan` 的 `VIntFixpAlu`	整数向量 ALU，按 SEW 切 lane 后逐元素相加

`Mgu` 是“掩码生成与合并单元”（Mask generation & merge unit）。它做两件事：根据 `vm`、`vl`、`vstart`、`ta`/`ma` 算出每个 lane“该不该写”，然后把新算出的值与旧值合并成最终的整寄存器数据。`VIAluFix` 在 `:148` 例化了 `Mgu`，在 `:333-345` 把 `vd`、`oldVd`、`mask`、`ta`、`ma`、`vl`、`vlmul`、`vstart` 接进去。这条指令的结果要过 `Mgu`，因为 `vm=0` 与 tail 策略都需要它。

数据通路：

```
v1 (128 bit) ─┐
              ├─▶ VIntFixpAlu (按 SEW 切 lane, 逐元素相加) ─▶ Mgu ─▶ res.data ─▶ 写回 v3
v2 (128 bit) ─┘                                              ▲
                                                             └ 旧 vd / v0 掩码 / ta / ma / vl
```

9.1 运算结果不会跨字节进位

RVV 规定 `vd[i] = vs2[i] + vs1[i]`，元素之间互不影响：第 i 个 byte 产生的进位不会进入第 i+1 个 byte。每个 lane 只取低 8 位结果，因此数值上等价于 16 个独立的 8 位加法。

这一点用实验验证过（`tests/hello/vadd_lane_isolation.c`，日志 `evidence/stage1/vadd-lane-isolation.log`）。做法是用 `vle8.v` 装载两个 16 字节向量、执行 `vadd.vv`、再用 `vse8.v` 存回并打印，构造“某一条 lane 刚好溢出、相邻 lane 有非零值”的输入：

用例	输入（前 3 字节）	结果（前 3 字节）	说明
A	`ff 00 00` + `01 00 00`	`00 00 00`	lane0 溢出，进位没有进入 lane1（否则 lane1 会是 `01`）
B	`11 ff 22` + `11 01 22`	**`22 00 44`**	lane1 溢出后，lane2 仍是 `44` 而不是 `45`——进位被限制在 lane1 内
C	`ff ff ff…` + `01 01 01…`	全 `00`	16 个 lane 同时溢出，各自归零、互不影响
D	`01 01 01…` + `01 01 01…`	全 `02`	常规相加，证明加法确实生效

B 用例是最能说明问题的一组：三个相邻 lane 都有值，中间那条溢出。如果实现中存在跨字节进位，lane2 必然变成 `0x45`；实测是 `0x44`，与逐 lane 手算一致。NEMU 与 RTL 的输出逐字节相同。

---

10. 写回与提交

时刻	信号	值
t=8569	io.out.valid	1
	data_0	0x0202…02
	pdest	51（v3 对应的物理寄存器）
	vecWen	1
t=8570	输出撤销	一拍有效

写回发生在 t=8569，结果写入物理寄存器并广播唤醒等待它的指令。提交在 ROB 按序完成，提交之后架构寄存器才对外可见。difftest 在提交处逐条比对，比对不通过会打印 different at pc=... 并中止。

---

11. 波形与流水线

11.1 各阶段在这条指令上发生了什么

阶段	这条指令的具体动作
Fetch	从 ICache 取出包含 0x021101d7 的指令块，经 FTQ/IBuffer 送入译码
Decode	匹配 VADD_VV 表项，产出 fuType=vialuF、fuOpType=0x40、vWen=1、三个向量源
Rename	v1、v2、v3 映射到物理寄存器，BusyTable 置位
Dispatch	uop 进入向量算术发射队列，ROB 分配表项（robIdx=24）
Issue	源操作数就绪后发出，进入 exus_2（VFEX2）
Register Read	同拍读出 src_0、src_1
Execute	VIntFixpAlu 完成 16 个 lane 的加法，结果过 Mgu
Writeback	out.valid=1，pdest=51，vecWen=1
Commit	ROB 提交，difftest 比对架构状态

11.2 波形原文

下面 0x01…01 表示 128 位整字都是 0x01（完整写法是 32 个十六进制位 0101…0101），0x02…02 同理。

```
t=8566/8567  io.in.valid = 0        # FU 空闲
t=8568       io.in.valid = 1        # 发射：uop 进入 VFEX2
             fuOpType    = 0x40     # 内部操作码，对应 vadd_vv
             vsew=0 vlmul=0 vm=1    # e8 / m1 / 不用掩码
             src_0 = 0x01…01        # v1：16 个 lane 全 1
             src_1 = 0x01…01        # v2：16 个 lane 全 1
             robIdx = 24            # 这条指令在 ROB 的位置
t=8569       io.out.valid = 1       # 结果有效，距进入 1 拍
             data_0 = 0x02…02       # 16 个 lane 全 2
```

```
              pdest  = 51            # v3 映射到的物理寄存器号
              vecWen = 1             # 写向量使能
t=8570        io.out.valid = 0       # 输出撤销，有效只保持一拍
t=8653        程序读到第一个字符 'v'  # 距 8569 共 84 拍（见 10.3 第 7 条）
```

11.3 波形怎么读

1. 先找 `io.in.valid` 的上升沿，那一拍就是指令进入 FU 的时刻（`t=8568`）。
2. 同一拍看 `fuOpType`，它对应译码阶段算出的内部操作码（`0x40` 即 `VialuFixType.vadd_vv`）。
3. 再看 `src_0`、`src_1`，确认源操作数（本例是两组全 1）。
4. 找 `io.out.valid` 的上升沿（`t=8569`），与进入时刻相差 1 拍，即 FU 的延迟。
5. 同拍看 `data_0` 与 `pdest`，确认结果值与写回目标。
6. `t=8570` 输出撤销，说明结果只在一拍内有效，写回逻辑必须在那一拍取走。
7. `t=8653` 是程序通过 UART 打印出第一个字符的时刻，与 `t=8569` 相差 84 拍。这段间隔由写回、提交、`vse8.v` 存内存以及打印代码本身构成，不属于这条加法指令的延迟。因此描述时要区分“FU 内 1 拍”与“结果多久被程序看到”。

---

12. 证据与复现

结论	证据文件
双端跑通、输出一致	`evidence/stage1/vadd-nemu.log`、`vadd-rtl.log`
字节间不互相进位	`evidence/stage1/vadd-lane-isolation.log`（源码 `tests/hello/vadd_lane_isolation.c`）
机器码与反汇编	上述日志中的 `objdump` 段落
逐拍信号值	`evidence/stage1/vadd-wave-cycle.txt`
波形原件	`evidence/wave/`

一条命令跑完（环境已就绪时）：

```
bash scripts/repro-stage1.sh
```

它依次做：环境自检 → hello 双端 → `vadd.vv` 双端比对 → lane 隔离 → 反汇编，结果写进 `evidence/stage1/repro-<时间戳>.log`。

手工对照（脚本里就是这些命令）：

```
# 编译
riscv64-linux-gnu-gcc -march=rv64gcv -mabi=lp64d -nostdlib -nostartfiles \
  -ffreestanding -O2 -fno-pic -fno-pie -mcmodel=medany \
  -T tests/hello/link.ld tests/hello/vadd.c -o /tmp/vadd.elf
riscv64-linux-gnu-objcopy -O binary /tmp/vadd.elf /tmp/vadd.bin
```

```
# 双端运行
timeout 20 ~/bin/nemu-standalone -b /tmp/vadd.bin
timeout 60 ~/xs-env/XiangShan/build/emu -i /tmp/vadd.bin --no-diff

# lane 隔离实验
riscv64-linux-gnu-gcc -march=rv64gcv -mabi=lp64d -nostdlib -nostartfiles \
  -ffreestanding -O2 -fno-pic -fno-pie -mcmodel=medany \
  -T tests/hello/link.ld tests/hello/vadd_lane_isolation.c -o /tmp/iso.elf
riscv64-linux-gnu-objcopy -O binary /tmp/iso.elf /tmp/iso.bin
timeout 60 ~/xs-env/XiangShan/build/emu -i /tmp/iso.bin --no-diff

# 查看机器码
riscv64-linux-gnu-objdump -d /tmp/vadd.elf | grep -B2 -A2 vadd
```